GAME DESIGN PORTFOLIO

시스템을 숫자로 검증하는 기획자

김태준

시스템 · 밸런스 기획

ktj4060@gmail.com · hellobot99.github.io/game-design-portfolio

Raid My Dungeon
01

Netrunner's Playlist
02

메이플 스타포스
03

ABOUT

기획한 것을 직접 만들고, 숫자로 확인합니다

게임 서버를 직접 구현하다가 시스템·밸런스 기획으로 방향을 잡았습니다. 기획서로 끝내지 않고 플레이 가능한 프로토타입과 계산 시트까지 만들어서, 의도한 숫자가 실제로 나오는지 확인하는 방식으로 일합니다.

직접 플레이로 검증

영웅 7명의 클리어 시간을 직접 재면서 스킬 보정을 다섯 번 고쳐 목표 범위에 맞췄습니다.

프로그래머 관점

C++ 서버를 직접 구현해 봐서 데이터 구조와 예외까지 생각합니다. NP에서는 코드와 데이터를 대조해 경험치 요구량이 역전되는 구간을 찾았습니다.

AI로 빠르게 만들고 확인

프로토타입, 시뮬레이터, 시트를 AI(Claude Code)로 만들고 결과는 직접 확인합니다. 스타포스 기대 비용은 10만 회 시뮬레이션으로 계산했습니다.

경북대학교 컴퓨터학부 (2027.07 졸업 예정) · 시바우라공업대학 교환학생 (2026.04 ~ 2027.01)

PROJECTS

시스템 · 밸런스 프로젝트 세 개와 AI 활용

01 RaidMyDungeon

**던전을 짓고
남의 던전을 터는 비동기 PvP**

기획 전담 · 5인 팀 · 2026.07 ~

플레이마다 얻는 자원을 나누고, 남의 던전을 터는 규칙과 악용 방지를 설계했습니다. 영웅 7명을 직접 플레이하며 클리어 시간을 맞췄습니다.

02 Netrunner's Playlist

사이버펑크 방치형 성장 게임

기획 + 서버(C++) · 2인 · 2026.01 ~

기존 데이터의 문제를 찾아 공식과 수치를 다시 설계하고, 30일 동안의 진행을 시뮬레이션으로 확인했습니다.

03 스타포스 역기획

메이플스토리 강화 시스템 역기획

개인 · 2026.10

규칙과 데이터 테이블을 개발 문서처럼 정리하고, 기대 비용과 불운을 계산해 개선안을 검증했습니다.

04 AI 활용

프로토타입과 검증을 빠르게

혼자 · 2026.07 ~

규칙과 숫자는 직접 정하고 코드는 AI로 만들어, 웹게임 네 개를 며칠씩 만에 플레이할 수 있게 했습니다.

01

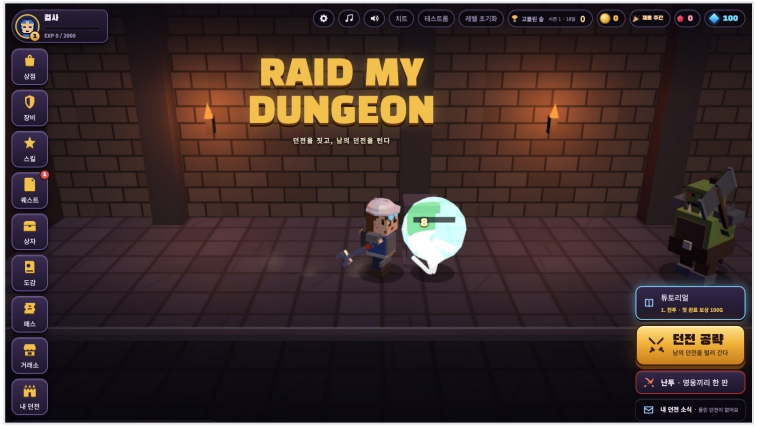
RAID MY DUNGEON

던전을 짓고,
남의 던전을 턴다

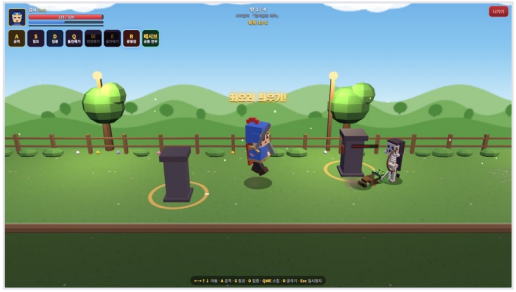
장르 액션 + 던전 빌딩
플랫폼 PC (웹 프로토타입)
기간 2026.07 ~
팀 5인
역할 기획 전담

던전을 짓고 남의 던전을 터는 비동기 PvP 게임

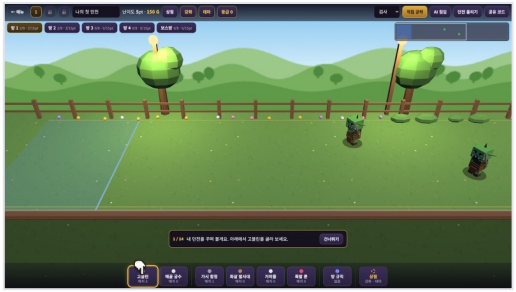
기획을 혼자 맡고 있습니다. 기획서를 시스템, 콘텐츠, 밸런스, 제작 네 갈래로 나눠 쓰고, 같은 수치로 돌아가는 웹 프로토타입과 밸런스 시트로 숫자를 확인했습니다. 클라이언트 2명, 모델링과 애니메이션 각 1명이 함께합니다.



웹 프로토타입 로비



남의 던전을 공략하는 화면



던전 편집기 (방마다 배치 예산)

19편
시스템 기획서

4편
밸런스 기획서와 엑셀

24건
이슈와 결정 기록

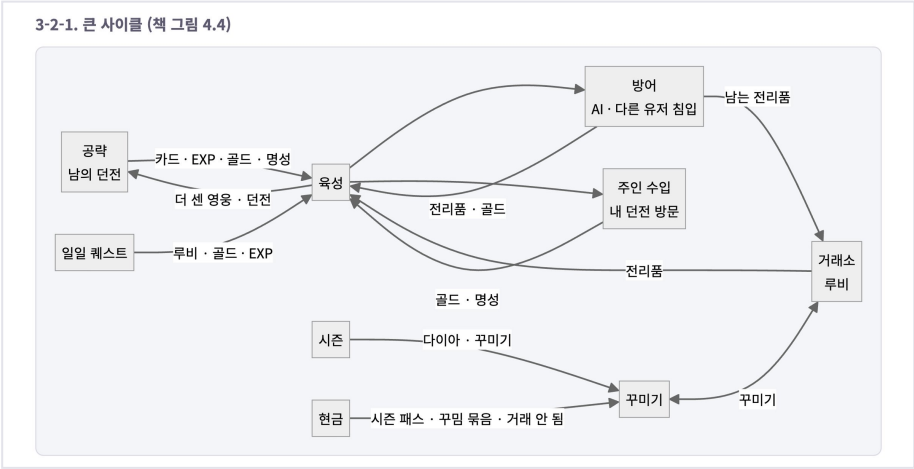
웹 기획서 웹 프로토타입

01

RAID MY DUNGEON

시스템 구조

플레이마다 얻는 자원을 하나씩 정해 두었습니다



경제 순환도 (책《게임 밸런스 수치 기획 바이블》 그림 4.4 형식)

거래는 거래소에서만

무엇을 어떤 화폐로 얼마나 거래할지는 시스템이 정한다. 던전을 잘 지어 남은 전리품을 팔면 루비가 되니, 던전을 잘 지을수록 돈이 된다.

플레이별 중심 자원

공략	카드
방어	장비 재료
주인 수입	골드
일일 퀘스트	루비
시즌	꾸미기

행동력 대신 자원 분리

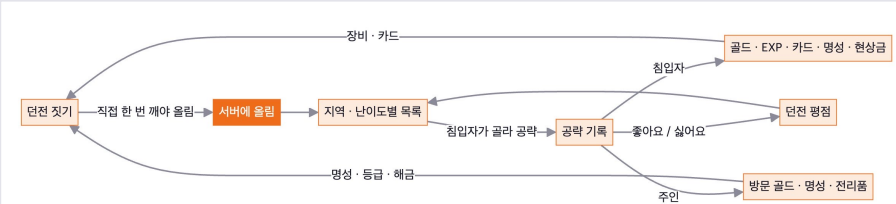
하루 횟수로 막지 않고, 한쪽만 하면 다른 쪽 자원을 못 얻게 했다

01

RAID MY DUNGEON

핵심 시스템
온라인 던전

남의 던전을 터는 규칙과 악용을 막는 장치



던전을 올리고, 털리고, 다시 짓는 순환

주인 수입
방문 수보다 적정 난이도, 접전, 조합, 평가로 정해진다

어려운 던전 보상
어려울수록 보상이 크고, 오래 안 깨진 던전에는 현상금이 붙는다

털렸을 때
주인은 잃는 것 없이 방문 보상을 받아서 부담 없이 공개한다

막아야 할 것	규칙
아무도 못 깨는 던전	주인이 자기 영웅으로 **직접 한 번 깨야** 올릴 수 있다
너무 쉬운 던전으로 방문료 벌기	방문 골드는 적게, 수입의 중심은 **적정 난이도 명성** (클리어율 30~70%에서 가장 큼)
친구끼리 돌려 깨기	같은 침입자는 그 던전에서 **하루 한 번만** 보상
들어왔다 바로 나가서 싫어요	방 2개 이상 들어가야 평가할 수 있다
결과 위조, 자동 사냥	서버가 공략 기록을 검사한다 (시간과 처치 수가 던전 내용과 맞는지, 사람이 낼 수 있는 기록인지)
오프라인 기록을 몰아서 보상	서버가 끊기면 편집과 연습만, 보상은 연결됐을 때만

01

RAID MY DUNGEON

시스템 기획서

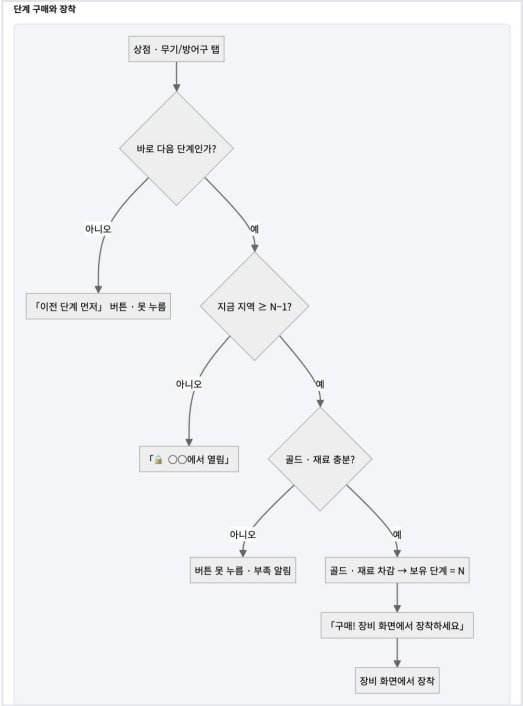
시스템 기획서는 개발자가 바로 옮길 수 있게 썼습니다

코드와 문서가 다르면 코드를 기준으로 하고, 숫자의 원본은 CSV에 두고, 결정은 날짜와 이유를 같이 남겼습니다.

절	장비 시스템 문서에서 쓴 것
1 목적	왜 있나, 어떤 시스템을 잇나
2 용어	단계 · 칸 강화 · 장비 재료
3 규칙	6단계 · 재료 6종 · 칸 강화 +0~10 · 능력치 공식
4 흐름	**흐름도 3개 (구매 · 강화 · 재료 순환)**
5 예외 처리	**모자람 · 잠김 · 최대 · 지역 하락 · 예전 저장**
6 UI	화면별 표시, 알림 문구
7 데이터	CSV 칼럼 · 저장 필드 · 코드 상수
8 연관 시스템	상점 · 재화 · 지역 · 온라인 던전
9 미정 사항	플레이 테스트로 볼 것

상황	처리
골드가 모자람	구매 버튼을 못 누름. 아무것도 차감하지 않음
지역에 아직 못 닿음	카드에 자물쇠, 「○○에서 열림」 표시
이전 단계를 안 삼	「이전 단계 먼저」 버튼, 못 누름
지역이 내려감	이미 열린 단계와 칸 상한은 닫히지 않음

위는 시스템 문서 19편이 같이 쓰는 9개 절, 아래는 장비 문서의 예외 처리 (일부)



장비 문서의 흐름도 (단계 구매와 장착)

01

RAID MY DUNGEON

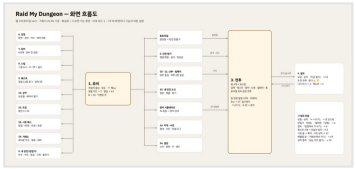
화면 흐름

화면마다 버튼의 기능과 이동할 곳을 적었습니다



1. **영웅 카드** — 얼굴 · 레벨 · EXP · **시즌 칭호 → 7. 장비**
2. **지역 · 명성** — 지금 지역 · 명성 · 「시즌 n · d일」 · 영웅 선택권 점 **→ 14. 지역 · 시즌**
3. **이벤트** — 주말 골드 축제 · 카드 / 재료 / 상자 주간 (07 §9)
4. **재화** — 골드 · 루비 · 다이아 (루비 = 일일 퀘스트 · 환전 · 거래소)
5. ⚙ **설정 → 16. 설정** · 옆에 음악 · 소리 음소거 · 치트 · 레벨 초기화 (개발용)
6. **왼쪽 메뉴** — 상점 · 장비 · 스킬 · 퀘스트 · 상자 · 도감 · **패스** · **거래소** · 내 던전 **→ 6 · 7 · 8 · 9 · 12 · 13 · 18 · 19 · 5**
7. **튜토리얼** — 전투 훈련장 → 던전 만들기 (편집기 코치)
8. **던전 공략 → 2. 던전 찾기**
9. **난투 → 10. 난투 · 11. 3:3 릴레이**
10. **내 던전 소식** — 몰린 던전 방문 · 받기 · 숫자 점 **→ 15. 내 던전 소식**
11. **침입자 방어 시뮬레이션** — AI 침입자가 내 던전을 공격 **→ 3. 전투 (방어 상자)**
12. **최근 소식** — 공략 · 상자 · 시즌 기록 · **거래소 팔림 · 주문 · 알림**

로비 화면 명세: 실제 화면에 번호를 달고, 버튼마다 기능과 이동할 화면(→)을 적음



화면 19개

왼쪽 전체 흐름도에서 화면 사이 이동을 먼저 정하고, 화면마다 이런 명세를 붙였습니다

01

RAID MY DUNGEON

밸런스 방법

책의 다섯 단계를 시트와 측정으로 옮겼습니다

책의 단계	정한 것	엑셀 시트	확인 방법
1 준비	전투 리듬, 생존 주기, 허용 오차	기준	
2 전투 수치	속성과 공식, 직업 보정, 몬스터	속성·공식, 직업 밸런스, 몬스터 템플릿	**딜량 측정, AI 공략 측정**
3 육성	성장 분할, 육성 주기, 카드 레벨	성장, 육성 플랜	**스킬 딜 비중 측정 (24명)**
4 경제	화폐, 소비, 생산, 가치	소비 모델	**하루 플레이 경제 시뮬레이션**
5 검증	직접 플레이, 1대1, 실측	플레이 검증, 결정 기록	**테스트룸에서 직접 플레이**

원칙: **직접 플레이 기록이 먼저**, 평타는 손대지 않고 스킬 보정으로, 모자란 만큼의 **70%만** 고치고 다시 측정, **합계를 먼저 정하고 나눈다**, 힘은 팔지 않는다

클래스	보스	전체	성공
검사	39	77.7	3/3
궁수	41.8	90.5	3/3
마법사	33.2	80.2	3/3
도적	29.6	66.3	3/3
격투가	31.7	80.9	3/3
성기사	32.2	69.2	3/3
거너	39.6	83.9	3/3

플레이 검증: 봇이 추천 던전을 실제로 공략한 시간 (초)

날짜	결정	값
10-03	허용 오차 · 원거리	±10% · 원거리도 근접과 같게
10-03	마법사 평타 예외	폭발 둘레 피해 70% → 40%
10-03	몬스터 수치	몬스터 HP ×2.5 · 보스 HP ×1.9
10-03	마법사 3차	스킬 보정 0.85 → 1.0
10-04	1차 7명 3차	검사 1.85 → 1.6 · 궁수 1.0 → 1.2

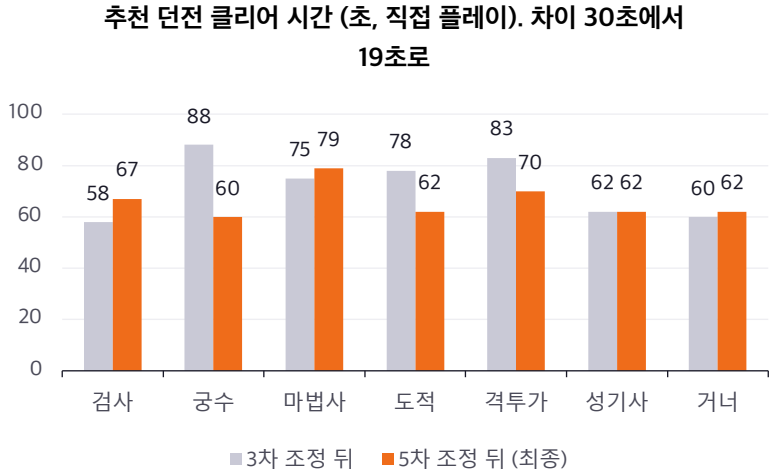
결정 기록 (일부): 보정값마다 날짜와 근거를 같이 남김

01

RAID MY DUNGEON

전투
밸런스

영웅 7명을 직접 플레이하며 다섯 번 맞췄습니다



최종 스킬 보정

검사	1.5
궁수	1.1
마법사	1.1
도적	1.3
격투가	1.0
성기사	0.45
거너	1.3

직업 역할
혼자 깨는 게임이라 딜은 같게, 플레이 방식만 다르게 했다

조정 대상
평타는 손맛이라 두고 스킬 보정만 고쳤다

조정 폭
모자란 만큼의 70%만 고치고 다시 측정했다

측정 기준
봇은 마법사 보호막을 못 살려서 직접 잰 기록을 썼다 (103초 → 79초)

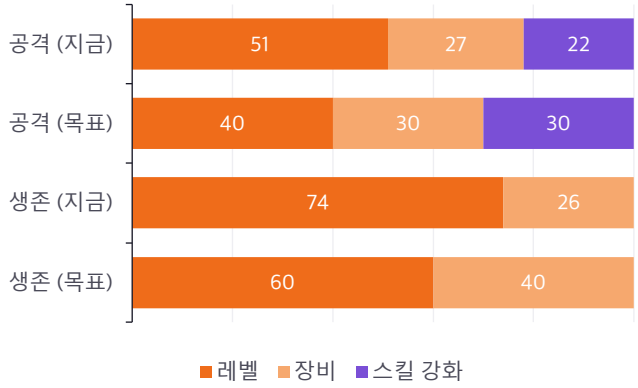
01

RAID MY DUNGEON

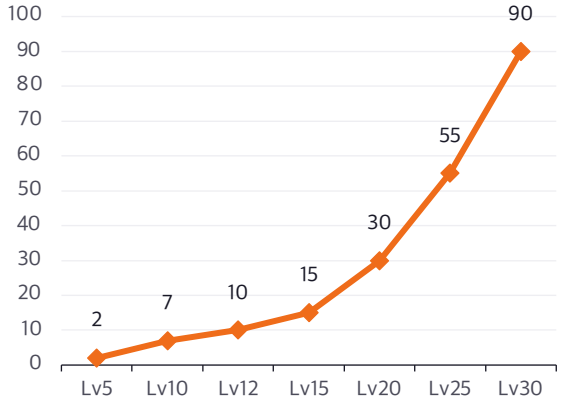
성장 밸런스

성장 총량을 먼저 정하고 경로마다 나눴습니다

경로별 성장 몫 (%)



레벨별 목표 도달 일수 (하루 30분~1시간)



나눈 이유

레벨 몫이 크면 경험치가 비싼 후반에 성장이 멈춘다. 그래서 골드로 사는 장비와 스킬 몫을 늘렸다. 장비는 모든 영웅이 같이 써서 새 영웅도 빨리 큰다.

스킬 몫 정하기

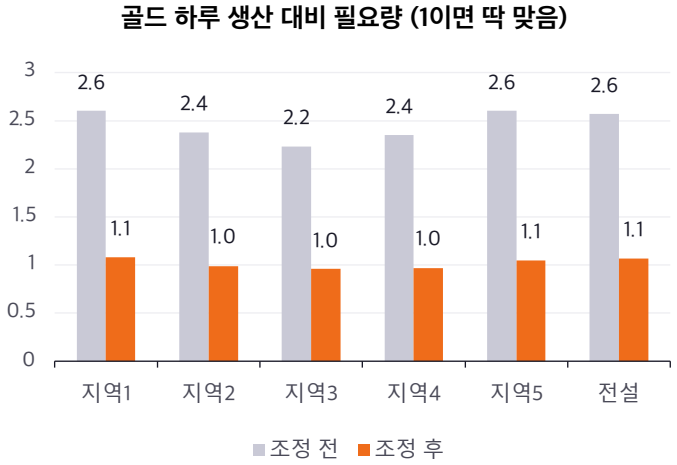
딜 중 스킬 강화가 붙는 몫을 영웅 24명 모두 재 보니 평균 57%. 그래서 스킬 칸마다 +14.3%로 정했다.

01

RAID MY DUNGEON

경제
밸런스

하루에 버는 양과 필요한 양을 맞췄습니다





다이아는 돈으로 팔지 않는다

유료 게임(Steam 4,900원) 안에 유료 화폐를 두지 않기로 했다. 시즌 패스와 꾸밈 묶음만 판다.

루비를 없애고 다이아로 거래하는 안도 봤지만, 값 단위가 너무 커져서 루비를 남겼다.

가격을 올림

보상 숫자는 그대로 두고 장비, 스킬, 카드 가격을 2.5배로

환전 한도

루비를 골드로 바꾸는 양은 하루 골드 생산만큼까지

가성비 흐름

한 단계 안에서는 떨어지고, 새 지역에서 다시 오르게

01

RAID MY DUNGEON

문제와 결정

찾은 문제와 정한 것을 문서로 남겼습니다

문제	정한 것
빈 던전 공유 코드로 보상을 무한히 받을 수 있음	같은 던전은 첫 클리어만 보상. 이후 **20pt 이상 + 보스**가 있어야 보상
연계 배율이 곱으로 쌓여 특정 순서만 피해가 폭증	연계 쪽 배율을 모두 곱한 값에 **상한 2.5배**
스킬 강화가 항목 고르기라 「피해부터」가 정답	**직선 강화**로 바꿈. 스킬마다 Lv1~8, 레벨마다 오르는 것 고정
레벨 몫이 커서 후반에 성장이 멈춘 느낌	성장 몫을 공격 레벨 40 · 무기 30 · 스킬 30, 생존 레벨 60 · 방어구 40으로 나눔
골드가 필요량의 2.5~3배 나옴	보상 숫자는 두고 **가격을 2.5배**로 (보상 받는 맛은 유지)
유료 게임 안에 유료 화폐를 둘지	**다이아를 돈으로 팔지 않음**. 루비를 없애는 안도 봤지만 값 단위가 너무 커져서 유지

문제를 찾고 정할 때마다 날짜, 이유, 바뀐 문서를 적어 두어서 팀원이 문서만 보고도 왜 이렇게 됐는지 알 수 있게 했습니다.

돌아보면 악용 가능성(공유 코드를 이용한 무한 보상 버그)을 직접 돌려 보고 나서야 찾았습니다. 규칙을 쓸 때 "어떻게 악용할까"를 예외사항 체크리스트로 먼저 점검해 보게 되었습니다.

02

Netrunner's Playlist

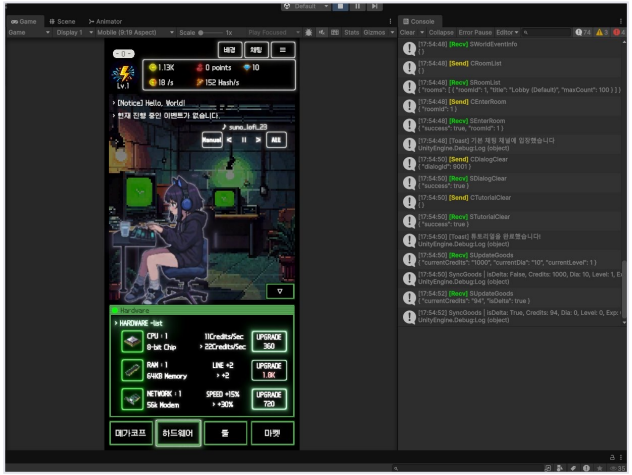
사이버펑크
방치형 게임

장르 방치형 성장
플랫폼 모바일 (Unity)
기간 2026.01 ~
팀 2인
역할 기획 + 서버(C++)

해커가 기업 서버를 뚫는 방치형 게임

기획과 서버를 맡았습니다. 기획서를 시스템과 밸런스 중심으로 다시 정리하고, 기존 데이터에서 문제를 찾아 공식과 수치를 새로 설계한 뒤 시뮬레이션으로 확인했습니다.





실제 게임 화면 (왼쪽은 클라이언트, 오른쪽은 직접 만든 서버의 패킷 로그)

서버 코드 (GitHub)

Lv51

경험치 요구량이 오히려 줄어드는 구간을 찾음

12시간

이전 지점 복귀 시간
(2회차는 48시간)

10회차

30일, 100티어까지 시뮬레이션으로 확인

02

Netrunner's Playlist

시스템 구조

성장 세 축이 돈(CPS)과 보스(DPS)로 나뉘어 들어갑니다

해킹 자동 + 클릭 → **크레딧 · 경험치** CPS · 오프라인 80% → **하드웨어 · 툴** CPU · RAM · NET → **메가코프 돌파** DPS 체크 20초 → **포맷(환생)** 악명 획득

성장 축	포맷 시	주는 재미	CPS(돈)	DPS(보스)
하드웨어 CPU · RAM · NET	1레벨로 초기화	숫자가 폭증하는 쾌감	○	○
해킹 툴 100종 · 슬롯 3~7	장착만 해제, 영구 소장	갈아 끼우는 전략	○	○
레벨 1 ~ 100	초기화 (시작 레벨 스킬)	단기 목표		○ 1.04^Lv
악명 + 악명 스킬 100	영구 누적	다음 회차를 빠르게		○ 회차마다 커지는 배율

보스 도전 → **20초 동안** 매초 HP −DPS · 클릭 −DPS 5% → **HP 0 → 다음 레이어** 최초 돌파 보상 · 점령 수입 교체 / **시간 초과 → 파밍 모드** 이전 단계에서 수입 쌓고 재도전

서버 검증	방식
보스 피해 조작	보고 피해 ≤ DPS × (경과 + 1) × 1.2
클릭 매크로	5초 배치 150회 초과 거절
오프라인 시간 조작	서버 시각 기준 · 12시간 상한

보스전 한 판의 흐름과 서버 검증

포맷하면
하드웨어와 레벨은 처음부터,
툴과 악명은 남는다
돈은 다시 벌지만 보스는 빨리 깬다

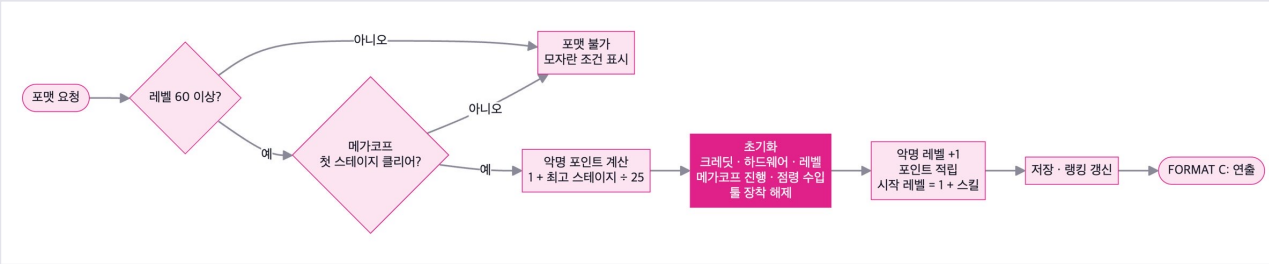
툴 슬롯은 레벨로 열림
포맷 직후엔 좋은 툴도 다 못 껴서
레벨을 다시 올릴 이유가 생긴다

02

Netrunner's Playlist

시스템 설계
포맷

포맷(환생) 때 지울 것과 남길 것을 나눴습니다



초기화	남는 것
크레딧 0	해킹 툴 (장착만 풀림)
하드웨어 1레벨	다이아
레벨 1 (+시작 레벨 스킬), 경험치 0	악명 레벨 · 포인트 · 스킬
메가코프 진행 · 점령 수입	업적

포맷 조건과 포인트 공식, 초기화 목록은 서버 코드 기준

나눈 이유
하드웨어는 다시 올리는 재미가 있어서 지운다. 툴은 모은 게 사라지면 허무해서 장착만 푼다.

예외 처리
조건이 모자라면 어떤 조건이 모자란지 보여 준다. 시작 레벨은 1에 악명 스킬만큼 더해, 툴 슬롯이 바로 열리게 한다.

02

Netrunner's Playlist

데이터 설계

데이터는 기획 표, 저장 데이터, 밸런스 로그로 나눴습니다

기획 데이터 (시트 → 서버 CSV)

- **megacorp** 7001~7500
  Tier · Layer · Theme · Maxhealth · Reward · CPS
- **tool** 3001~3100
  effect · value · Method · Price · Grade
- **skilltree** (악명 스킬) 5001~5100
  Effect · Value · Cost
- **achievement** 6001~6100
  EventTrigger · CheckKey · TargetValue · Reward
- **CPU · RAM · Network** 1~ · 1001~ · 2001~
  Level · Name · Visual State
- **exp · dailydungeon · worldevent · cps_boost**
  8001~

유저 저장 데이터 (DB)

- **users**
  credits · dia · level · exp · infamy_level · infamy_points · last_logout_time
- **user_hardware**
  cpu_level · ram_level · network_level
- **user_tools / user_equipped_tools**
  tool_id → tool.ID · slot_index
- **user_megacorp**
  current_stage_id · max_cleared_id → megacorp.ID
- **user_skills / user_achievements**
  skill_id → skilltree.ID

밸런스 로그 (분석용)

- **balance_cps_log**
  레벨별 최종 CPS · 배율
- **balance_warning_log**
  효율 상한 · 소프트캡 도달
- **player_levelup_log**
  레벨업 시점 · 보상
- **megacorp_clear_log**
  클리어 시간 · DPS · 보스 HP
- **tool_transaction_log · credit_tracking_log**
  툴 획득 · 크레딧 흐름

ID 앞자리로 종류를 나눠 **한 번호만 보고 어느 표인지** 알 수 있게 했다. 유저 데이터는 기획 데이터의 ID만 들고, 수치는 늘 표에서 읽는다 (표만 고치면 전체 반영)

서버 저장 구조와 로그 테이블은 직접 설계하고 구현했습니다. 로그는 레벨별 CPS와 상한 도달을 모아 숫자가 의도대로 가는지 보는 데 씁니다.

02

Netrunner's Playlist

공식 ·
서버 구현

같은 하드웨어를 돈과 보스에 다른 계수로 씁니다

CPS 평소 해킹 → 크레딧

```
11·CPU × (1 + RAM/2) × (1 + 0.15·NET)
× (1 + 크레딧 보너스, 상한 +300%)
× (1 + 0.5% × 점령 티어)
전체 배율 10배 넘으면 로그로 증가
오프라인: × 0.8 · 최대 12시간
```

DPS 보스전 20초 → 보스 HP

```
120·CPU × (1 + 0.1·RAM) × (1 + 0.05·NET)
× 1.04 ^ (레벨 − 1)
× 악명 배율 (×1.25, ×1.55, ×1.85 …)
× (1 + 섹터 피해) × (1 + 보스 피해)
클릭 1회 = DPS × 5%
```

부품마다 계수와 비용을 다르게 둔 이유: 셋 다 같은 때 같은 돈으로 올리면 고르는 재미가 없어서, 실제 장비 가격을 참고해 부품마다 느낌을 다르게 했다. 레벨과 악명은 **DPS에만** 곱해서, 포맷하면 돈은 처음부터지만 보스는 빨리 깬다

```cpp
// User::CalculateCPS() · 서버 (C++)
double toolCpuEff = toolBonus.hwCpuEff;
if (toolCpuEff > cpuCap) {            // 툴 효율 상한
    LogManager::Instance().LogBalanceWarning(playerId_, level_,
        "CPU_EFF_CAP", toolCpuEff, cpuCap, ...);
    toolCpuEff = cpuCap;
}
...
if (totalMultiplier > multiplierCap) {  // 10배 이후 로그 증가
    totalMultiplier = multiplierCap
        + std::log10(totalMultiplier − (multiplierCap − 1.0));
    LogManager::Instance().LogBalanceWarning(..., "LOG_SCALING", ...);
}
```

직접 구현한 서버의 CPS 계산 (상한에 닿으면 밸런스 로그를 남김)

기획자가 숫자를 확인하는 장치

툴 효율과 크레딧 보너스에 상한, 전체 배율은 10배부터 완만하게

상한에 닿을 때마다 **balance_warning_log**

레벨별 평균 CPS를 뽑아 보는 분석 쿼리

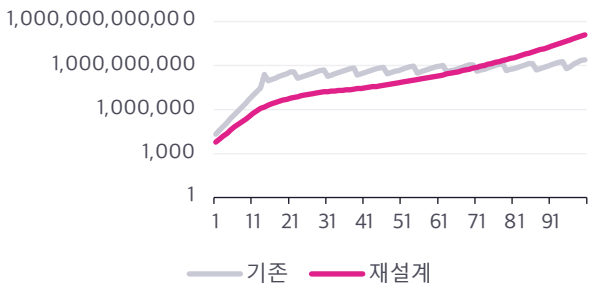
02

Netrunner's Playlist

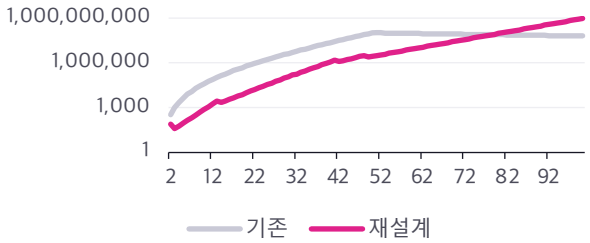
기존 데이터 진단

코드와 데이터를 대조해 수치 문제를 찾았습니다

티어별 보스 HP (로그 축). 기존은 톱니 모양



레벨별 필요 경험치 (로그 축). 기존은 51부터 감소



기존 데이터의 문제	재설계
경험치: 첫 포맷까지 84시간, 51레벨부터 필요량이 **줄어듦**	목표 시간에서 역산, 계속 증가
레벨업 보상: 레벨마다 ×1.4라 Lv100에 **7.2경**	그 레벨 CPS 3분치
악명 스킬 RAM_ADD 3,000으로 DPS가 수천 배 (DPS엔 상한 없음)	ADD는 하드웨어 레벨 +1로
보스 HP: 8티어마다 떨어지는 톱니, 14티어만 튐	목표 DPS × 20초 × 레이어 계수
점령 수입: 100티어도 후반 CPS의 0.3%	티어당 CPS +0.5%
악명 스킬 총량: 다 찍으려면 포맷 약 400회	약 15회
툴 가격: 희귀 799억, 영웅 800조	해금 시점 CPS 1 · 3 · 8시간치

서버 코드와 데이터 테이블을 대조해 찾은 16건 중 일부

찾은 방법

서버 공식과 데이터 시트를 직접 읽고, 목표 CPS로 레벨 도달 시간을 계산

02

Netrunner's Playlist

수치 재설계

목표 플레이 시간에서 거꾸로 수치를 정했습니다

기준값 > **성장 구축** > **성장 분할** > **세분화** > **검증**

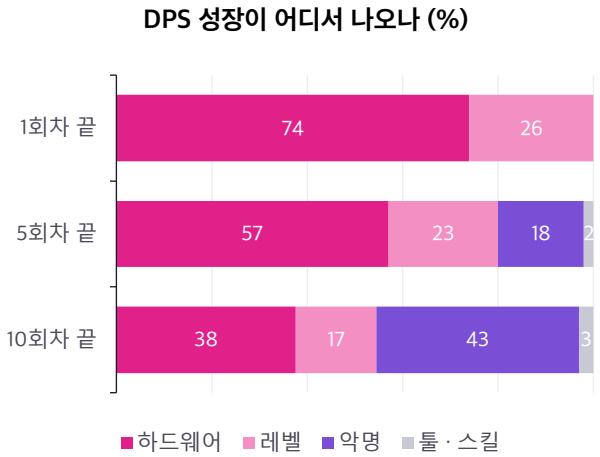
하루 2번 20분
1회차 80시간, 30티어

접속마다 목표 DPS

하드웨어, 레벨,
악명, 툴

데이터 표 8종

10회차
시뮬레이션



회차 안에서는 하드웨어가, 회차를 넘어서는 악명이 성장을 맡는다

바뀐 공식

하드웨어 비용	1만, 3만, 2만 × 1.3^Lv
레벨 계수	1.04^(Lv-1)
악명 배율	×1.25, ×1.55, ×1.85 순으로 커짐
보스 HP	목표 DPS × 20초 × 레이어 계수
보상 · 가격	CPS 몇 분 · 몇 시간치
오프라인	80% · 12시간 (오프라인 위주)

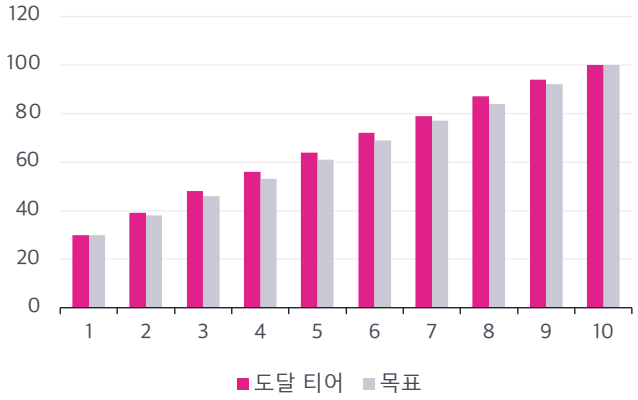
02

Netrunner's Playlist

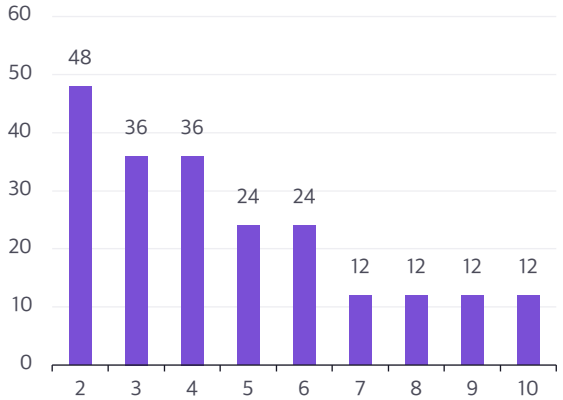
시뮬레이션
검증

30일 동안 포맷 10번을 시뮬레이션으로 확인했습니다

회차별 도달 티어



이전 최고 지점까지 복귀 시간 (h)



1회차

72시간에 30티어, Lv60
첫 포맷 조건을 채움

복귀

48, 36, 24, 12시간으로
회차마다 짧아짐

10회차

100티어 도달
악명 누적 ×1,936

수입

오프라인이 86%
방치 중심 설계

가정: 하루 2번(오전 8시, 오후 8시) 20분 접속, 접속 중 초당 2회 클릭, 구매와 보스 도전은 접속 중에만. 툴과 스킬은 회차마다 평균 효과(CPS ×1.1, DPS ×1.06, 시작 레벨 +3)로 계산

02

Netrunner's Playlist

밸런스 시트

기준값 몇 개로 500개 스테이지가 계산되는 시트

기준값 (노란 칸을 바꾸면 다른 시트가 다시 계산됨)

공식과 수치 초안 v3 · 시뮬레이션: sim/np_sim.py

하드웨어		
CPU 기본가	10,000	RAM ×3 · NET ×2
RAM 기본가	30,000	
NET 기본가	20,000	
비용 배수 (레벨당)	1.30	Lv → Lv+1 비용 = 기본가 × 배수^Lv
전투		
레벨 계수 밑 (DPS × 밑^(Lv-1))	1.04	
보스 제한 시간 (초)	20	
레이어 1 계수	0.30	
레이어 2 계수	0.45	
레이어 3 계수	0.60	
레이어 4 계수	0.80	
레이어 5 계수 (보스)	1.00	
31티어 이후 티어당 성장 시작값	1.050	31티어 ×1.05 → 99티어 약 ×1.23
31티어 이후 성장 가속 (티어마다)	0.0023	
악명		
악명 1레벨 배율	1.25	악명 k레벨 배율 = 1.25 + 0.3 × (k-1)
레벨마다 배율 증가	0.30	
경제		
오프라인 효율	80.0%	오프라인 위주 방향
오프라인 상한 (시간)	12	
점령 수입 (티어당 CPS %)	0.5%	30티어 +15%, 100티어 +50%

기준값 시트: 노란 칸을 바꾸면 10개 시트가 전부 다시 계산된다



요약 시트 (바뀐 공식, 1회차 검증, 차트)



ID	이름	티어	산업 분야	레이어	목표 DPS	HP	최초 돌파 보상	점령 수입 (CPS +%)
7001	NovaShop	1	Finance	1	300	1,800	240	0.1%
7002	NovaShop	1	Finance	2	300	2,700	240	0.2%
7003	NovaShop	1	Finance	3	300	3,600	240	0.3%
7004	NovaShop	1	Finance	4	300	4,800	240	0.4%
7005	NovaShop	1	Finance	5	300	6,000	1,440	0.5%
7006	TechSphere	2	Finance	1	479	2,874	520	0.6%
7007	TechSphere	2	Finance	2	479	4,311	520	0.7%
7008	TechSphere	2	Finance	3	479	5,748	520	0.8%
7009	TechSphere	2	Finance	4	479	7,664	520	0.9%

메가코프 시트 500행 (모두 수식)

돌아보면

처음엔 목표 플레이 시간 없이 데이터를 손으로 넣어 보스 HP가 톱니 모양이 됐습니다. 목표 시간부터 정하고 공식으로 표를 만드니 더 좋은 밸런스를 찾을 수 있었습니다.

03

메이플스토리 스타포스

강화 시스템
역기획

대상 상용 라이브 게임
범위 스타포스 강화
형식 개발 전달용 기획서
기간 2026.10

강화 한 번을 규칙, 데이터, 숫자로 다시 썼습니다

메이플스토리 스타포스를 개발자에게 넘기는 기획서처럼 다시 정리했습니다. 규칙과 예외를 표로 쓰고, 데이터 테이블을 설계하고, 기대 비용과 불운을 계산한 뒤 개선안을 숫자로 검증했습니다.

1
규칙과 플로우
강화 결과와 예외

2
데이터
테이블과 규칙 대조

3
수치 분석
기대 비용과 불운

4
비교
로스트아크, 던파

5
개선안
확률 슬라이더

194억

250제 장비 0 → 22성
평균 메소 (파괴 2.65회)

463억

불운한 상위 1%
평균의 2.4배, 불운 보정 없음

129억

개선안 적용 후 불운 격차
(지금 269억의 절반 아래)

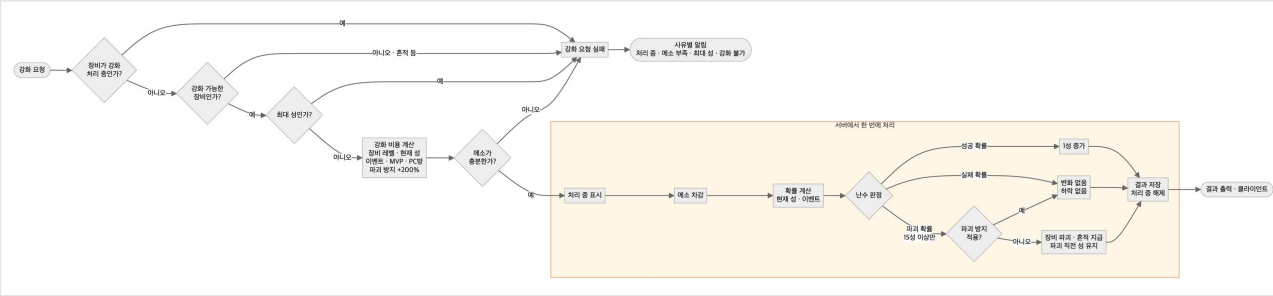
계산 전에 감으로 적은 값은 80억 · 파괴 1회였습니다. 실제로는 2.4배였고, 1회 비용이 아니라 "1회 비용 ÷ 성공 확률"로 봐야 한다는 걸 이 차이에서 배웠습니다.

03

메이플스토리 스타포스

규칙 ·
플로우 차트

판정 전에 거를 것과 한 번에 처리할 것을 나눴습니다



강화 1회 플로우. 주황 상자 안(처리 중 표시 ~ 결과 저장)은 서버에서 한 번에 처리해 메소만 빠지는 일이 없게 했습니다

결과를 바꾸는 요소	결과	예외 상황	처리
현재 성	성공 · 실패 · 파괴 확률. 15성 미만은 파괴 0	**이미 강화 처리 중**	요청 거절 (버튼 연타 · 중복 요청)
장비 레벨	비용 공식, 최대 성, 스탯 증가	**흔적을 강화하려 할 때**	강화 대상에 못 올림. 흔적은 복구에만
파괴 방지 (15 ~ 17성)	파괴 → 실패. 비용 +200%	**메소 부족 · 최대 성**	버튼이 눌리지 않고 사유 알림. 메소는 빠지지 않음
이벤트	비용 할인, 5 · 10 · 15성 100%, 파괴 확률 감소	**결과 전에 게임 종료**	서버가 받았으면 차감 · 판정 · 저장까지 한 번에 끝냄
슈페리얼 장비	확률 · 비용 따로, 흔적은 0성	**이벤트가 끝나는 순간**	**서버가 요청을 받은 시각**으로 판단

판정 전에 거르는 4가지
처리 중 → 흔적 → 최대 성 → 메소

한 번에 처리
차감, 판정, 저장을 묶어서 끊겨도 메소만 빠지지 않음

시각은 서버 기준
이벤트 경계는 서버가 받은 시각으로

03

메이플스토리 스타포스

데이터
테이블 설계

값이 바뀌는 때와 기준으로 테이블을 나눴습니다

같은 표에 넣으면 한 줄이 무엇인지 흐려져서, 무엇으로 한 줄을 찾는지(키)와 언제 바뀌는지로 나눴습니다. 3장 규칙 18개가 전부 어느 표의 어느 칸으로 표현되는지 대조했습니다.



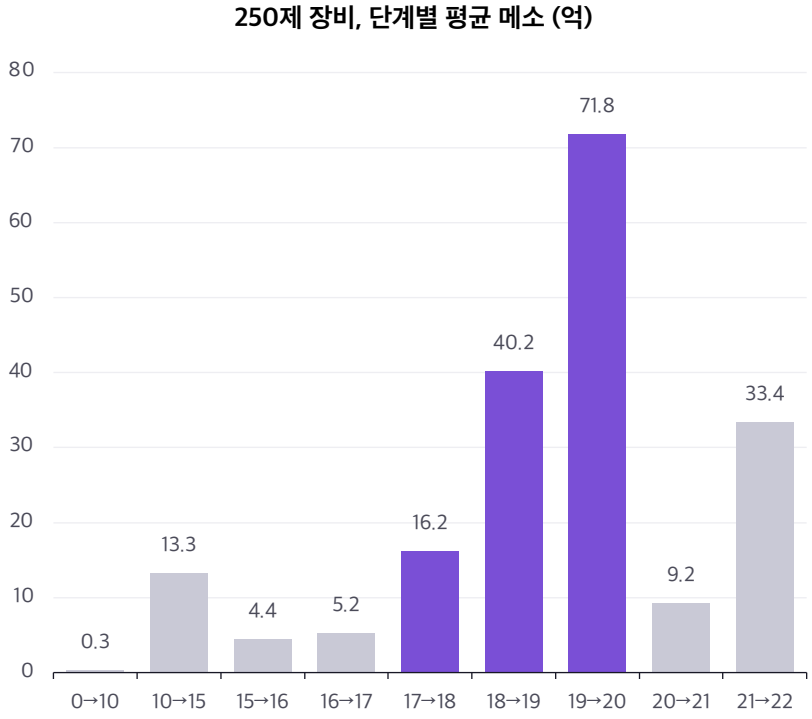
표로 남지 않은 규칙 3개는 서버 처리 규칙으로 따로 적음: 파괴 방지 시 파괴 → 실패, 할인은 파괴 방지 비용에 적용 안 함, 차감 · 판정 · 저장은 한 번에

03

메이플스토리 스타포스

수치 분석

22성까지 드는 메소의 3분의 2가 세 단계에 몰립니다



194억

평균 · 시도 60회 · 파괴 2.65회

128억

17 → 20성 세 단계 (66%)

304억 · 463억

불운 상위 10% · 상위 1%

11배

공격력 1당 메소, 16→17성 대비 19→20성

단계마다 성공 확률이 p면 평균 시도는 1/p (하락이 없어 기하분포). 분산은 시뮬레이션 10만 회. 확률 15 ~ 29성은 2025.03.20 공식 공지, 0 ~ 14성 확률 · 비용 계수 · 스탯은 커뮤니티 정리 값 (역기획서 부록)

03

메이플스토리 스타포스

다른 게임
비교

스타포스는 불운에 끝을 두지 않는 쪽을 골랐습니다

	메이플스토리 스타포스	로스트아크 재련	던전앤파이터 강화
실패 시 손실	메소. 15성부터 파괴, 흔적이 파괴 직전 성 유지	재료만. **하락 · 파괴 없음**	고단계에서 하락 · **파괴**
파괴 막기	파괴 방지 (17성까지, +200%)	해당 없음	장비보호권: 파괴 대신 +0
불운 보정	**없음 · 매번 독립 확률**	**장인의 기운**: 실패가 쌓이면 다음은 100%	확인 못함
재화	메소 하나	골드 + 여러 재료	골드 + 큐브 조각
거래	강화된 장비를 경매장에서	장비는 캐릭터 귀속	확인 못함

불운의 끝이 있나
로스트아크: 성공률 10% 단계라도 약 22번 실패하면 다음은 반드시 성공
스타포스: 끝이 없음. 15.75% 단계에서 30번 연속 실패할 확률 0.6%

그 선택의 장단점
장점: 메소를 크게 태우고, 고성 장비의 희소성이 유지됨
단점: 같은 22성이 운에 따라 2배 넘게 차이 남 (178억 vs 463억)

두 게임은 직접 플레이하지 않고 공식 가이드 · 공개 자료로 비교. 던파 일부 항목은 확인하지 못함

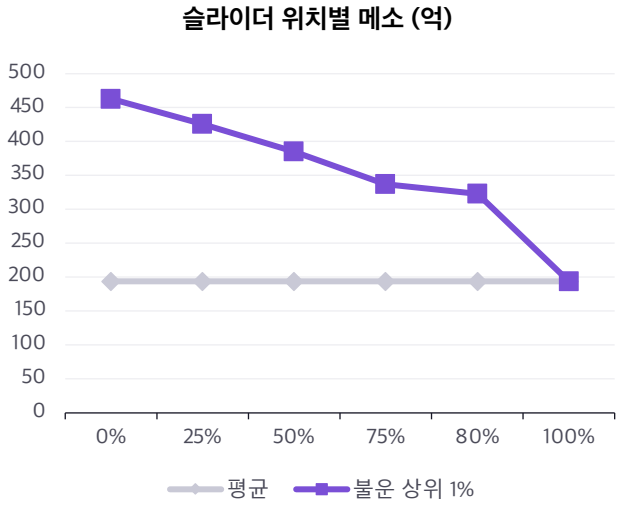
03

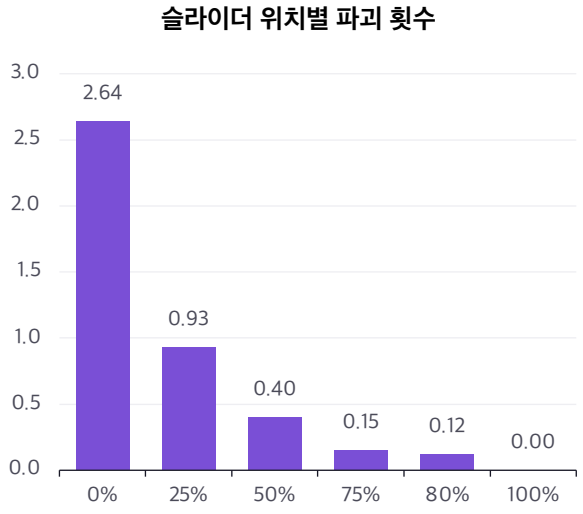
메이플스토리 스타포스

개선안 · 전후 검증

평균은 그대로 두고 불운 격차를 절반으로 줄였습니다

확률 슬라이더를 당긴 만큼 성공 확률과 1회 비용을 같은 비율로 올립니다. 끝까지 당기면 기대 비용 그대로 확정 성공입니다. 평균 메소 소각은 그대로 두고, 불운 격차(상위 1% – 평균)는 절반으로, 파괴는 1회 아래로 줄이는 게 목표였습니다. 1.x배 메소를 받고 대신 강화해 주는 "대장장이"가 유행하는 걸 보고, 돈을 더 내더라도 불운을 피하고 싶은 수요가 있다고 봤습니다.





80%에서 격차 129억 (지금 269억), 25%만 당겨도 파괴 0.93회. 웃돈을 붙이면 조금 당긴 유저가 손해라 웃돈 없이 설계. 확률만 올리는 안은 ×3에도 상위 1%가 12%만 줄어 제외

04

AI 활용

작업 방식

도구 Claude Code
범위 프로토타입 · 문서 · 시뮬레이션 · 검증
기간 2026.07 ~

아이디어를 며칠 만에 플레이할 수 있게 만듭니다

규칙과 숫자는 직접 정하고, 코드와 반복 작업은 AI에 맡겼습니다. 결과는 직접 플레이하고 측정해서 판단합니다. 아래 네 단계를 프로젝트마다 반복했습니다.

기획 결정 직접 | **구현 · 반복 작업** AI | **플레이 · 수치 확인** 직접 | **기획서 · 기록 반영** 직접

작업	AI가 한 것	직접 한 것
프로토타입	RMD 웹 프로토타입, 웹게임 4개의 코드와 배포	규칙, 수치, 플레이해 보고 고칠 곳 판단
기획 문서	RMD 웹 기획서와 화면 흐름도의 형식과 그림	화면 목록, 버튼 기능, 결정 내용
시뮬레이션	NP · 스타포스 · BattleCoach 시뮬레이터 코드	조건과 목표를 정하고 결과를 해석
검증	공략 봇, 페이스 봇, 자동 점검 스크립트	봇이 못 잡는 건 직접 측정

04

AI 활용

작업 과정

AI와 만들면서 직접 내린 결정들

Riddle404 · 방향과 제약

“실시간으로 계속 API를 부르지 말고, 사건을 만들 때만 쓰자”

→ AI는 사건과 문장만 만들고, 이름 · 테마 · 방 구조는 직접 만든 목록에서 무작위로 뽑게 했다. 마지막 보고서용 호출도 처음 받은 결과를 쓰게 해서 없앴다.

Riddle404 · 플레이하다 찾은 구멍

“5명이 다 현장에 없다고 나오는 시각을 찾았는데, 여기서 어떻게 범인을 추리하지?”

→ 소리 기록을 둘로 나눠, 가짜 시각엔 다섯 명이 다 보이고 진짜 시각엔 범인만 안 보이게 바꿨다. 노트가 답을 너무 많이 알려 줘서 정리 기능만 남겼다.

BattleCoach · 결과 의심하기

“기술 데미지 임의로 정한 거 아니지?” “퍼즐 난이도는 어떻게 조정하고 있어?”

→ 기술 수치가 쇼다운 배틀 엔진에서 나오는 것을 확인했다. 퍼즐은 테마별 관문 대신 1단계부터 쭉 오르게 바꾸고, 시뮬레이션 승률로 난이도를 맞췄다.

The Last Harbor · 컨셉 다시 잡기

“아직 게임이라기보다 시뮬레이터 느낌. 디테일이 아니라 시스템을 바꿔야 할 듯”

→ 판타지 무역 RPG에서 대항해시대로, 다시 땅따먹기에 무역을 돈벌이로 붙인 구조로 바꿨다. 그 뒤 영역 확장이 너무 빨라 확장 속도를 늦췄다.

따옴표 안은 작업할 때 Claude Code에 실제로 보낸 질문입니다. 답을 받은 뒤 어느 쪽으로 갈지는 직접 골랐습니다.

04

AI 활용

웹게임 4개

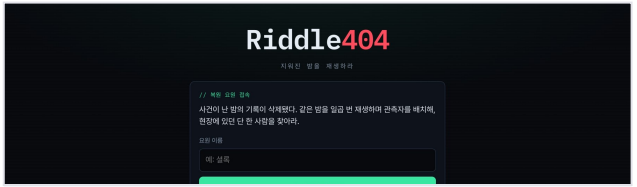
2026년 10월, 혼자 웹게임 네 개를 만들었습니다



BattleCoach 하루 플레이

포켓몬 배틀을 하며 턴마다 코치가 수를 평가하는 학습 게임

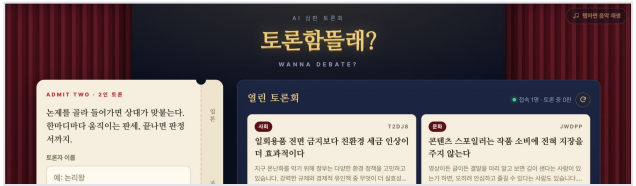
직접 정한 것 9단계 평가 등급, 퍼즐 100단계 난이도



Riddle404 1주 플레이

같은 밤을 일곱 번 재생하며 범인을 찾는 1~6인 협동 추리

직접 정한 것 단서 구조, 난이도 5단계, 인원 보정



토론함뜰래? 이틀 플레이

AI가 주제를 내고 심판을 보는 2인 실시간 토론

직접 정한 것 진행 단계와 시간, AI 채점 기준



The Last Harbor 열흘 플레이

대항해시대 실시간 땅따먹기와 무역

직접 정한 것 무역 경제, 데이터 표 6종, 페이스 봇

그 밖에: 도서관 데이터 공모전 웹 구현(팀), 친구들과 하려고 만든 Cobblemon 서버의 아이템 가격과 배틀 팩토리 상점 조정

04

AI
활용

BattleCoach
난이도

퍼즐 100단계를 목표 승률에 맞춰 골랐습니다

단계마다 목표 승률(1단계 93% → 100단계 30%)을 먼저 정하고, 그 승률이 나오는 팀 조합을 시뮬레이션으로 찾았습니다. 난이도는 상대 AI, 팀 전력, 도움 세 가지를 함께 올렸습니다.

묶음	상대 AI	팀 전력	도움
1~20	센 기술 위주, 실수 25→0%	내 팀이 확실히 강함	힌트 무제한
21~40	불리하면 교체	조금 강함	힌트 8번 → 5번
41~60	불리하면 교체	비슷함	상성·속도 표시 끔
61~80	코치와 같은 평가기	상대가 조금 강함	힌트 3번 → 2번
81~100	코치와 같은 평가기	상대가 확실히 강함	턴 45초, 힌트 0번

한계도 적어 뒀습니다
시뮬레이션의 플레이어는 사람이 아니라 AI라 체감은 다르다. 가장 센 AI는 너무 느려 한 단계 아래로 대신 돌려서, 61단계부터는 표보다 실제로 더 어렵다.

1 후보 만들기
전력 차에 맞는 팀 조합 24개

2 싸워 보기
후보마다 30판, 목표에 가까운 4개는 100판씩

3 고르기
목표 승률에 가장 가까운 조합

4 다시 찾기
5%p 넘게 벗어난 단계는 후보를 3배로

100단계 중 95단계

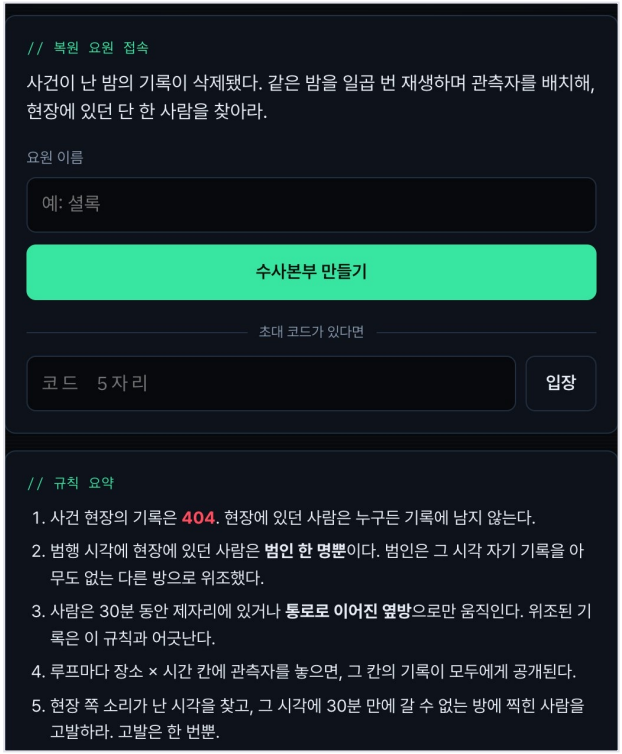
목표 승률 ±5%p 안. 가장 먼 단계도 6%p

04

AI 활용

Riddle404
규칙 설계

범인이 한 명으로 좁혀지는 추리 규칙을 짰습니다



riddle404.onrender.com

핵심 규칙

장소 6곳 × 시간 8칸의 지도에서, 같은 밤을 일곱 번 재생하며 칸마다 관측자를 둔다
범행 시각 현장에는 범인 한 명뿐이고, 현장 기록은 지워져 있다
소리 기록 두 개 중 하나는 가짜. 가짜 시각엔 다섯 명이 모두 다른 방에 보이고, 진짜 시각엔 범인만 안 보인다
인원 보정: 팀 전체 관측 수를 고정해 적은 인원도 불리하지 않게

단계	더해지는 규칙
1 기본	없음
2 공범	위조 기록에 공범이 함께 찍혀 거짓 증언
3 정전	한 시각 동안 사람 수만 남음
4 잠긴 문	통로 하나가 잠김. 범인을 가리는 단서가 그 문
5 이중 위조	위조가 두 칸이라 단서가 한 칸 더 바깥

서버가 사건을 만든 뒤, 어떤 단계에서도 관측만으로 범인이 한 명으로 좁혀지는지 확인한 사건만 낸다.

04

AI 활용

판단 사례

불운 격차를 줄일 안을 시뮬레이션으로 골랐습니다

1 상황
스타포스 22성까지 평균 194억, 운 나쁜 상위 1%는 463억

2 목표
평균 메소 소각은 그대로 두고 불운 격차(상위 1% – 평균)를 절반으로

3 안 나누기
확률 업, 천장, 확정 구매, 확률 슬라이더 네 가지

4 계산
안마다 10만 회 시뮬레이션으로 평균과 상위 1%를 비교

5 판단
확률 업은 격차가 거의 그대로라 버리고, 평균은 그대로 두면서 격차가 절반이 되는 슬라이더를 골랐다

안	평균	상위 1%	결과
지금	194억	463억	
확률 업 (비용도 ×3)	196억	409억	버림 · 격차 거의 그대로
천장 (기대 비용 1.5배)	160억	266억	버림 · 소각 18% 감소
확정 구매 (절반 쓰면)	204억	323억	보류 · 유저 행동에 달림
확률 슬라이더 80%	**194억**	**323억**	**채택 · 격차 절반**

250제 장비 0 → 22성 기준. 시뮬레이터와 계산은 AI로 돌렸고, 비교할 안과 목표, 버릴지 말지는 직접 정했습니다.

04

AI 활용

검증

AI가 만든 결과는 직접 측정해서 확인했습니다

RaidMyDungeon · 봇 대신 사람 기록
공략 봇은 마법사 보호막을 살리지 못해 103초가 나왔다. 직접 플레이로 재 보니 79초. 스킬 보정은 사람 기록으로 정했다.

BattleCoach · 평가 기준 다듬기
코치 카드에 "이 평가가 이상해요" 버튼을 두고, 누르면 그 턴의 상황을 저장해 등급 기준을 고치는 데 썼다.

The Last Harbor · 자동 점검과 페이스 봇
숫자를 바꿀 때마다 무작위 자동 플레이로 병력과 금화가 깨지지 않는지, 같은 명령이 같은 결과를 내는지 확인했다. 페이스 봇으로 몇 분에 끝나는지 쟀다.

Netrunner's Playlist · 고칠 것은 직접
AI로 서버 코드와 기획서를 대조해 어긋난 곳을 찾고, 51레벨부터 경험치가 줄어드는 역전처럼 무엇을 먼저 고칠지는 직접 정했다.

AI가 낸 숫자와 코드는 시작점으로만 쓰고, 플레이와 측정으로 맞는지 확인한 뒤에 기획서에 반영했습니다.



감사합니다

Email
ktj4060@gmail.com

Portfolio
hellobot99.github.io/game-design-portfolio

GitHub
github.com/Hellobot99